Model Validation

Gian Marco Miccio

ReLU

22/09/2025

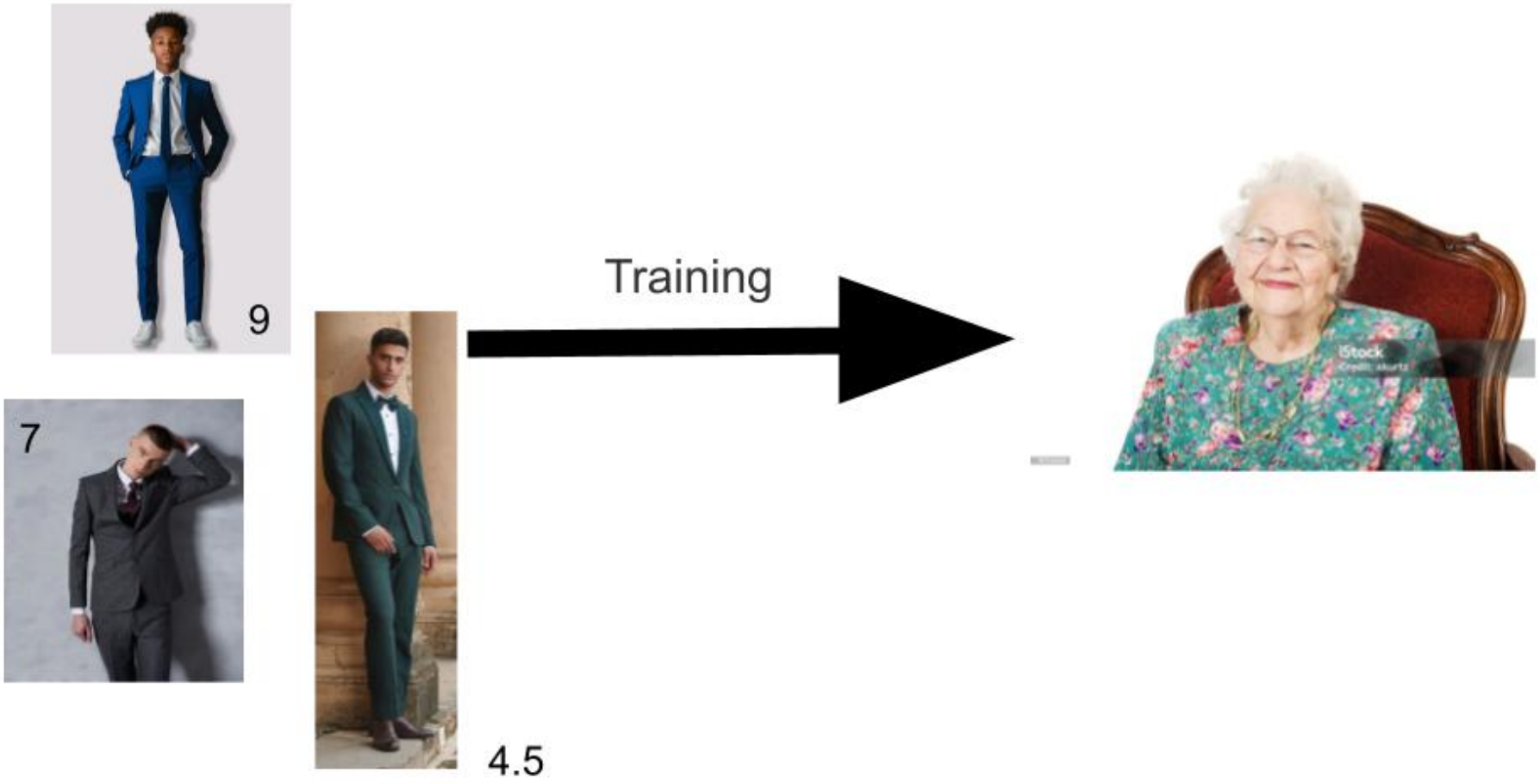
Let's go through a conceptual example, and to prove that these concepts can be applied to anything...

Meet your Machine Learning models:

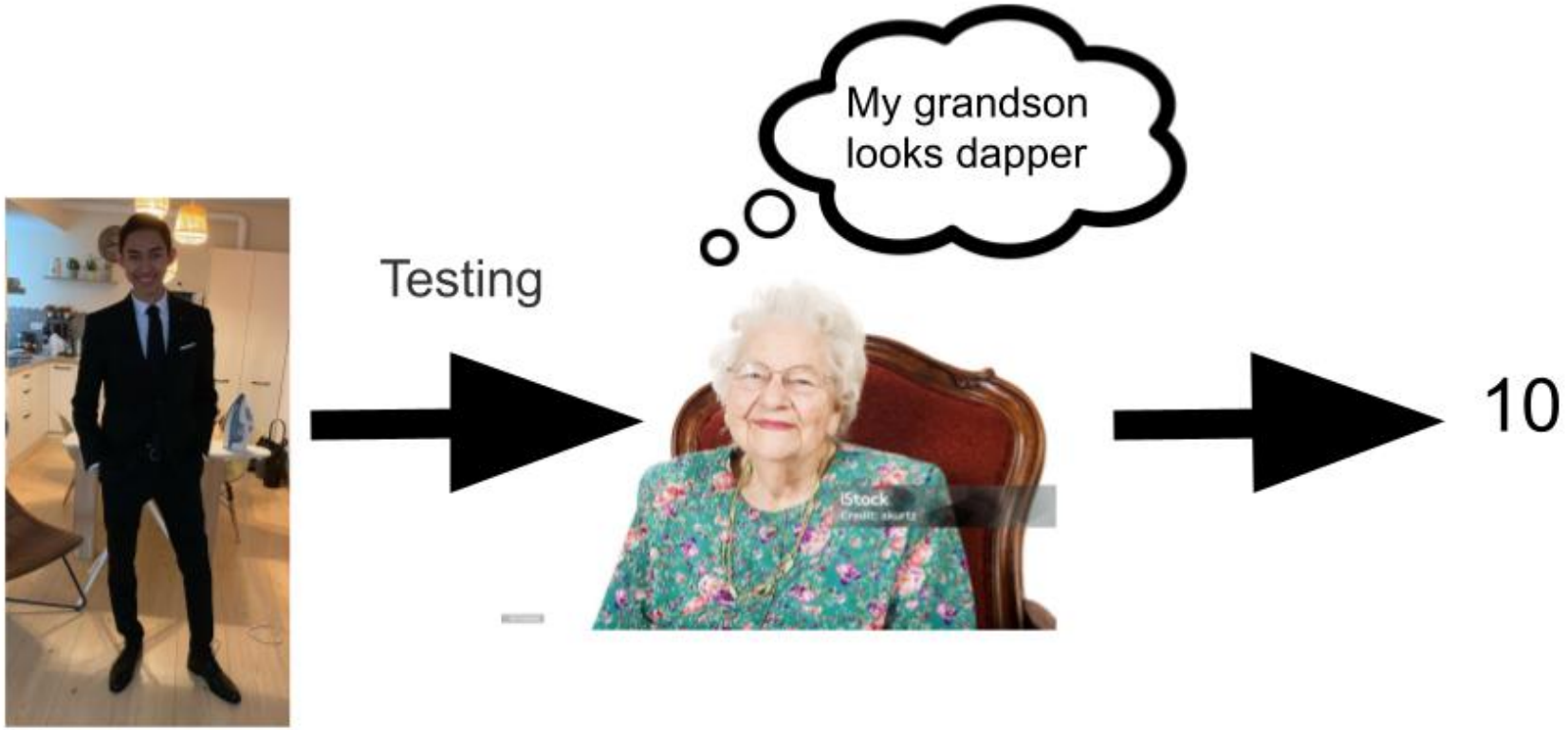
Your Grandma and your Dad

ML problem: you want to figure the best attire to go to prom

You train your Grandma with some pictures of your what your friends are going to wear



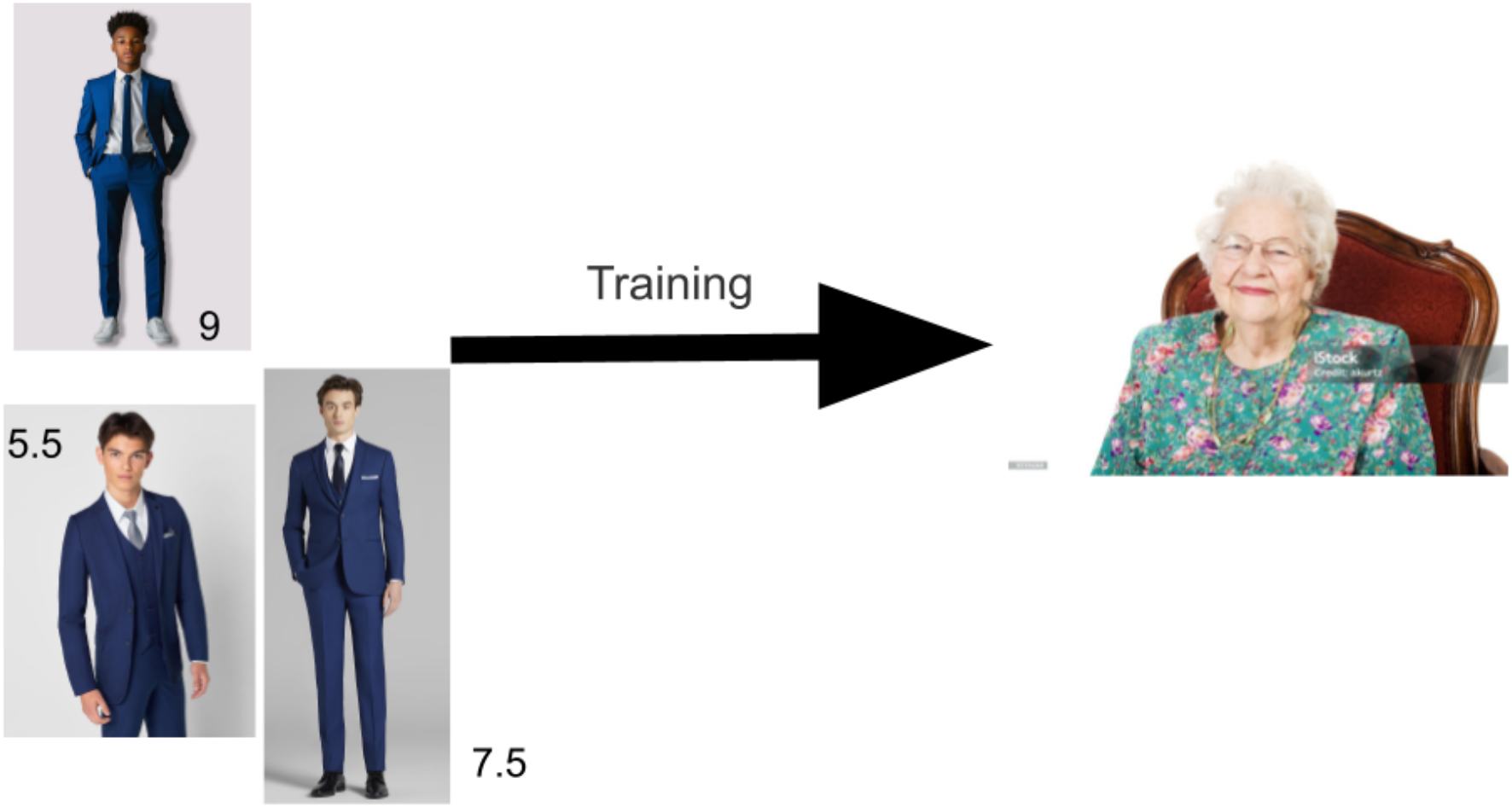
You then test your grandma by showing her your suit



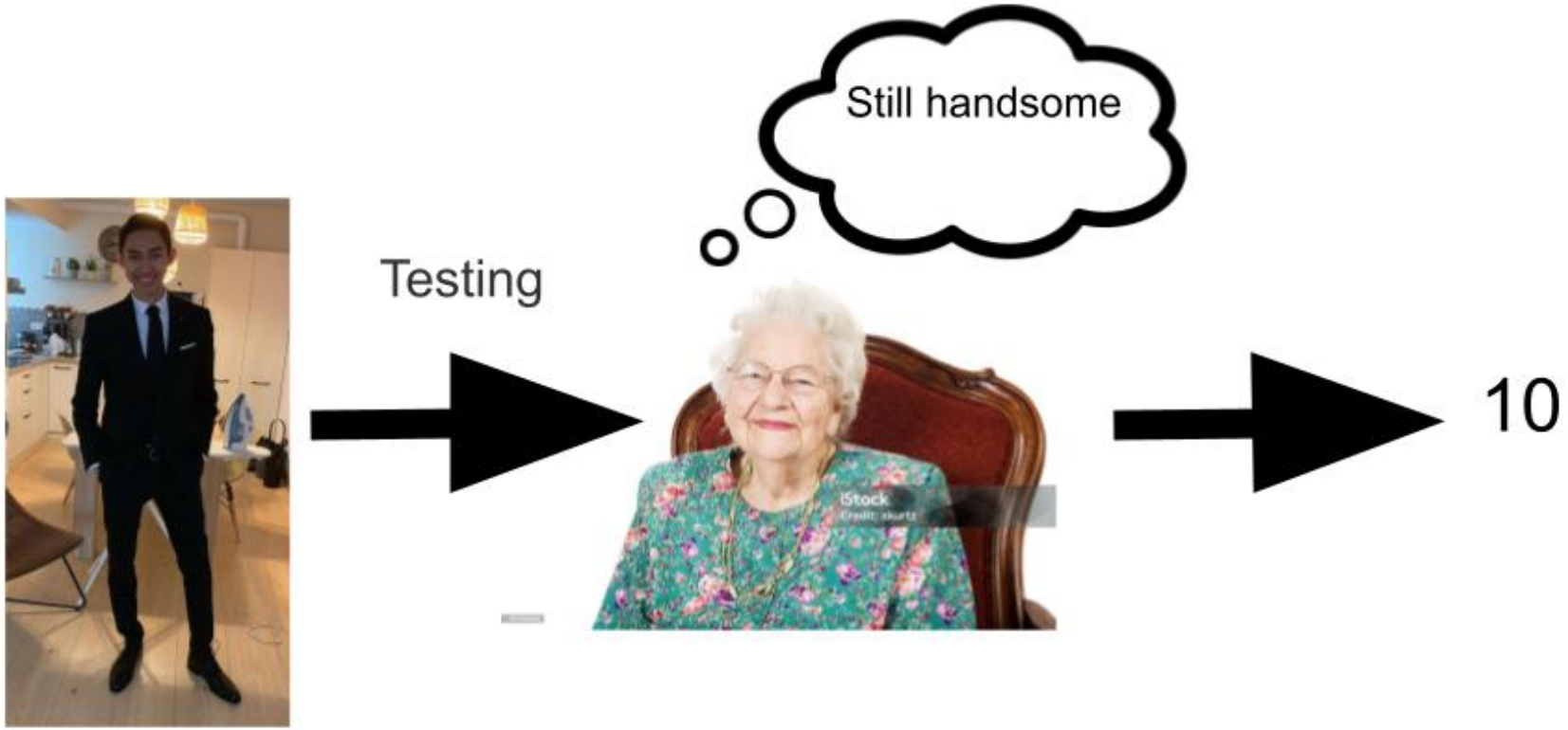
You now train your Dad with the same images

And then you test him

What if you train your Grandma on a different dataset?



She still gives you a 10



Your Grandma is a **biased** model, no matter how you train her she will always bump your rating up

You train your Dad on another dataset

Your Dad now thinks your initial suit sucks because everyone is wearing blue and you are not

Your Dad is a **high variance** model, because based on the training it will give very different result

Bias vs Variance

We've looked at this conceptual example, now let's try to generalized this

Bias vs Variance

Let's take a look at the elements of our framework:

Consider y to be the **observed values**, $f(x)$ the model that represents the **true relationship** between our observed values and the set of **covariates** x that we chose for our model, and an **error** ε

$$y = f(x) + \varepsilon$$

Also, any **proposed model** we can think is represented by $\hat{f}(x)$, and this is because in practice we are not going to guess the **true relationship** $f(x)$

$$y = \hat{f}(x) + \varepsilon$$

Bias vs Variance

Thought Experiment: Imagine you were trying to predict the price that a house is going to be sold at

What kind of covariates could you think of collecting to create a model to predict this?

Bias vs Variance

Any covariate you thought of would in your case be added to the dataset **x**, while any other you haven't considered would directly contribute to ε and its variance.

Remember: there's always going to be a covariate you haven't thought of/couldn't measure

Key Idea: no matter how many covariates we introduce, how much we tweak the model, there's always going to be an **irreducible error** ε

$$y = \hat{f}(x) + \varepsilon$$

Bias vs Variance

How do we compare proposed models $\hat{f}(x)$ between each other?

Enter the **Mean Square Error**

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right]$$

Idea: it's the squared difference between our proposed model $\hat{f}(x)$ and the true observed value y

Bias vs Variance

Important: When we looked at linear regression, the data $\mathbb{X}$ was **fixed**, and given that we were doing linear regression.
Now instead we are taking a step backwards, the data x (as we will call it in this case) is **variable**.

Idea: you can think of it as fitting the same model to study for the same phenomenon, but with data measured on a different day, by a different person, in a different environment

Bias vs Variance

Important: it can be decomposed in the following elements

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \underbrace{VAR[\varepsilon]}_{\text{Irreducible error}}$$

Bias vs Variance

Proof

Expand the square expression

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \mathbb{E}\big[(f(x) + \varepsilon - \hat{f}(x))^2\big]$$

$$= \mathbb{E}\big[\underbrace{(f(x) - \hat{f}(x))^2}_{\text{what we care about}}\big] + \underbrace{\mathbb{E}\big[2\varepsilon(f(x) - \hat{f}(x))\big]}_{0} + \underbrace{\mathbb{E}[\varepsilon^2]}_{\sigma^2}$$

Bias vs Variance

Introduce $\mathbb{E}[\hat{f}(x)]$ to the difference, and we expand the terms

$$(f(x) - \hat{f}(x))^2 = \left(f(x) - \mathbb{E}[\hat{f}(x)] + \mathbb{E}[\hat{f}(x)] - \hat{f}(x)\right)^2$$

$$= (\text{f(x)-E[(x)]})^2 + 2(f(x) - \mathbb{E}[\hat{f}(x)])(\mathbb{E}[\hat{f}(x)] - \hat{f}(x)) + (\mathbb{E}[\hat{f}(x)] - \hat{f}(x))^2$$

Proof

The expectation of the middle element is:

$$\mathbb{E}\Big[2(f(x) - \mathbb{E}[\hat{f}(x)])(\mathbb{E}[\hat{f}(x)] - \hat{f}(x))\Big]$$

$$= 2(f(x) - \mathbb{E}[\hat{f}(x)])\underbrace{\mathbb{E}[\mathbb{E}[\hat{f}(x)] - \hat{f}(x)]}_{=0} = 0$$

because

$$\mathbb{E}[\mathbb{E}[\hat{f}(x)] - \hat{f}(x)] = \mathbb{E}[\hat{f}(x)] - \mathbb{E}[\hat{f}(x)] = 0.$$

Proof

Hence the cross term vanishes, leaving:

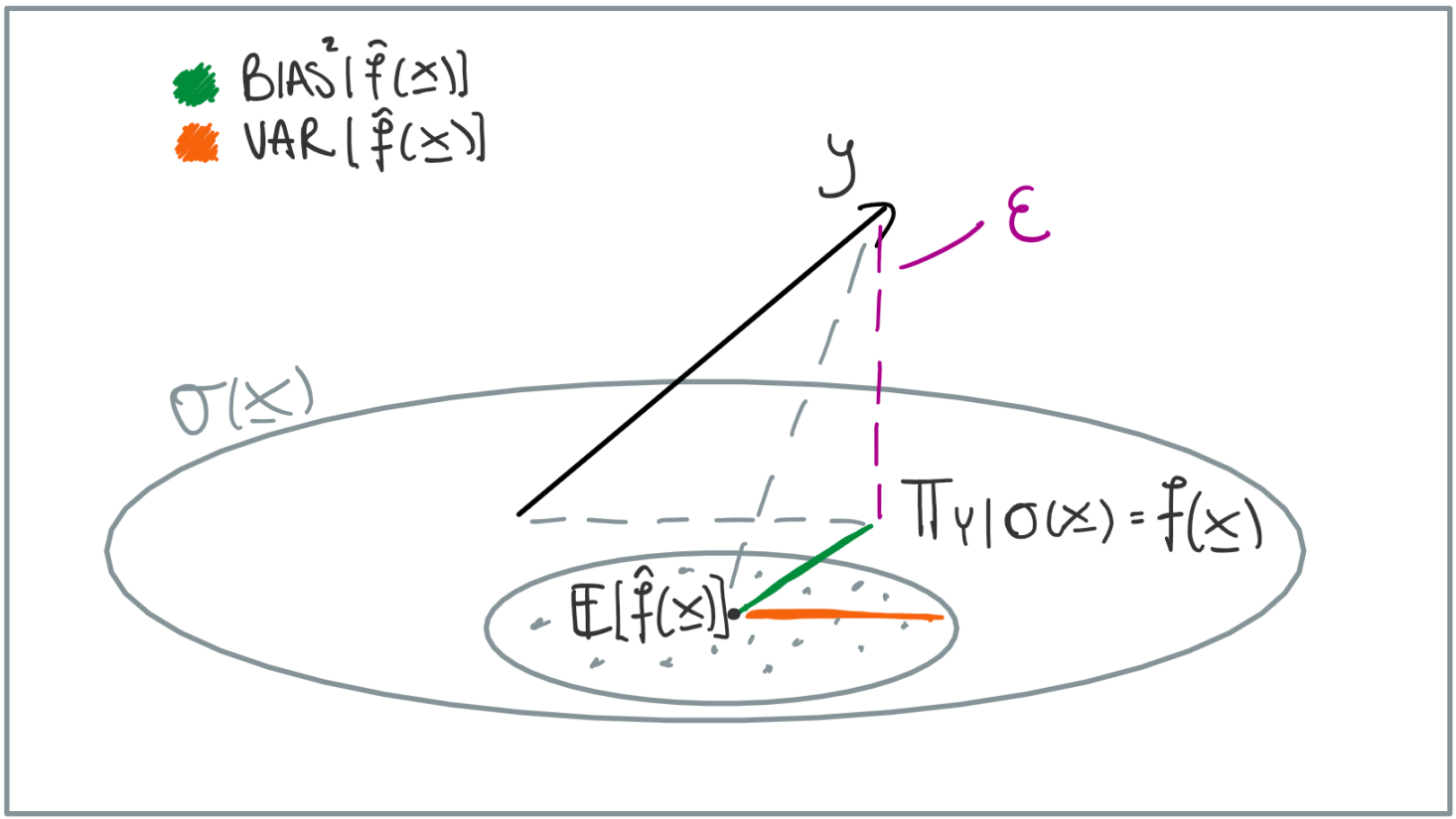
$$\mathbb{E}[(f(x) - \hat{f}(x))^2] = (f(x) - \mathbb{E}[\hat{f}(x)])^2 + \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2].$$

Combine

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \underbrace{VAR[\varepsilon]}_{\text{Irreducible error } \sigma^2}$$

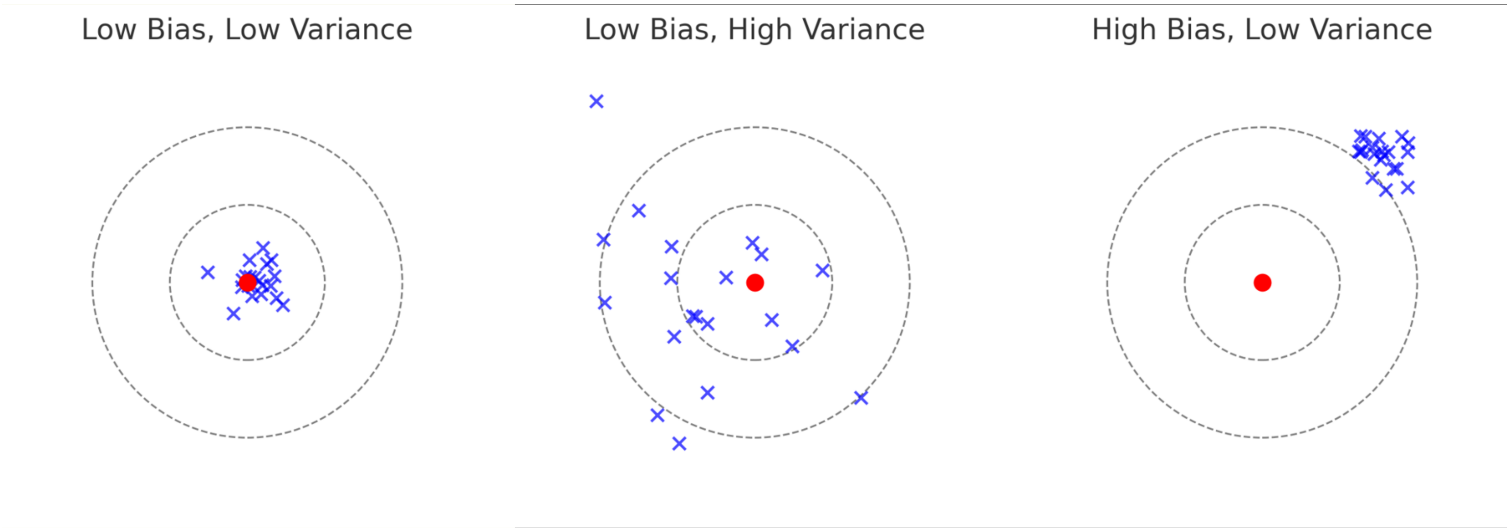
Bias vs Variance

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}$$



Bias vs Variance

To help you visualize further:



Idea

- **Model Variance** indicates how variable the model to change in data
- **Model Bias** indicates how close on average the model will be to the truth

Bias vs Variance

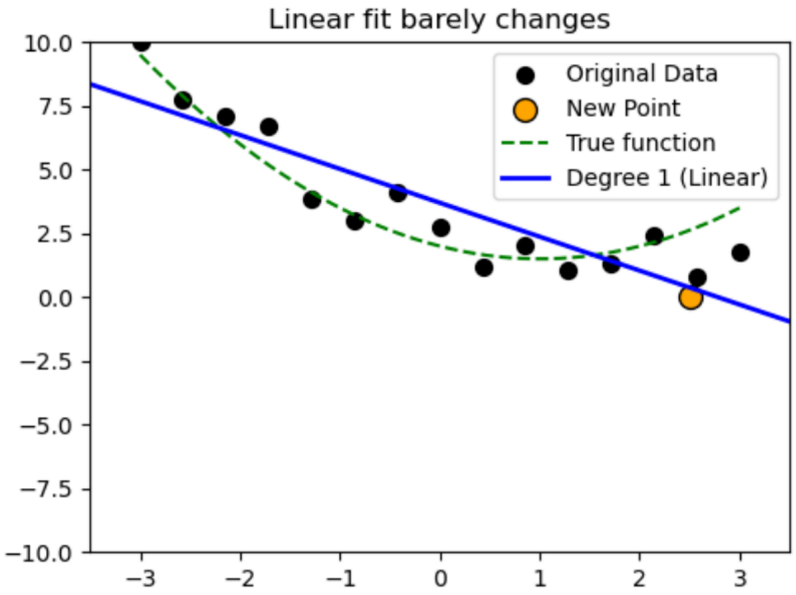
Consider an example of Polynomial Regression, meaning instead of fitting a line we want to fit an entire polynomial

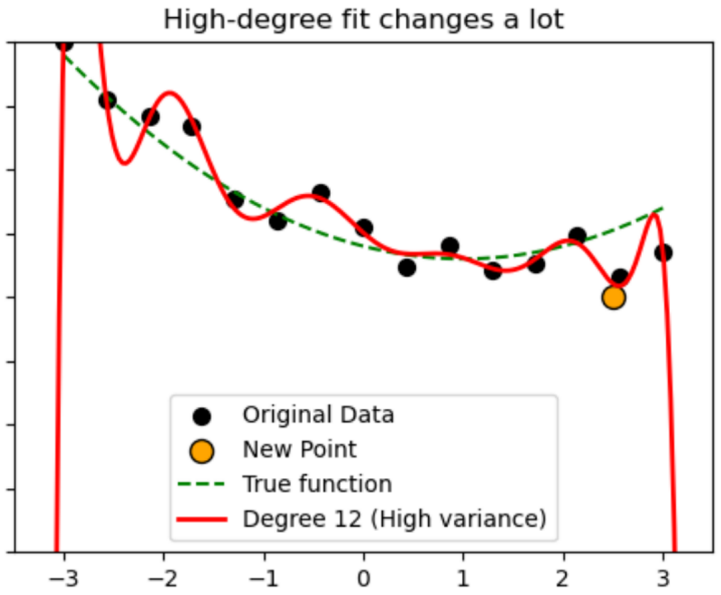
$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d + \varepsilon$$

if $d = 1$, then we are looking at linear regression

Bias vs Variance

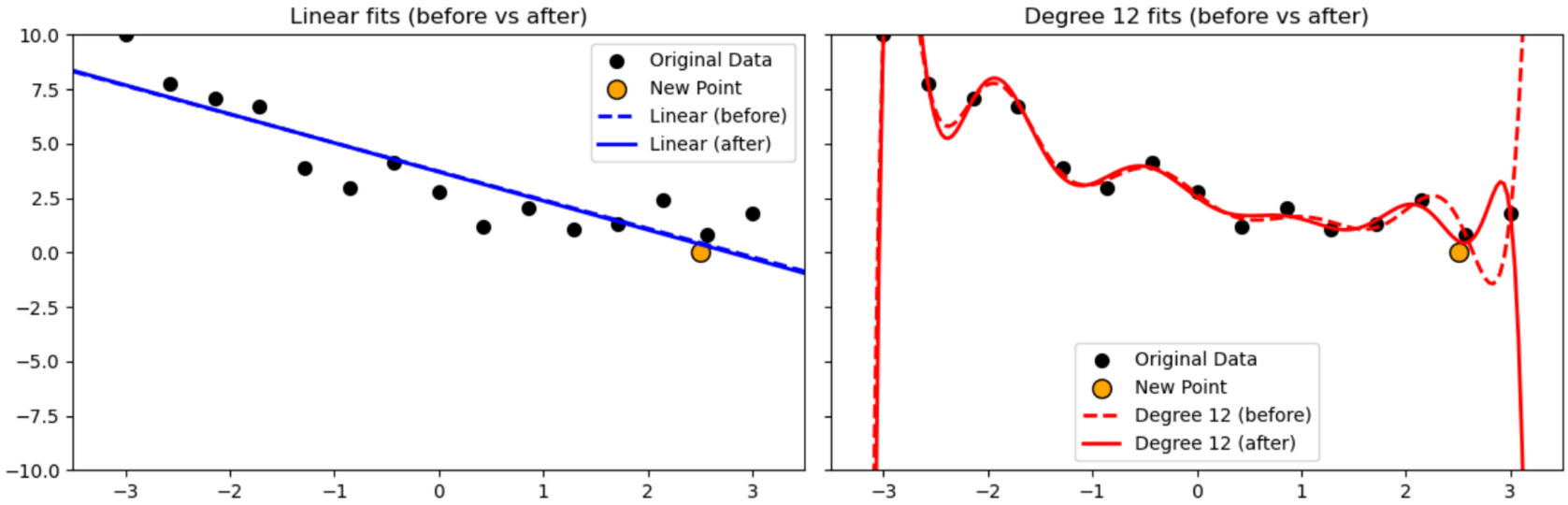
Let's look at an example where we know the true model follows a polynomial of degree $d = 2$, and let's see what happens if we fit two regressors of $d = 1$ and $d = 12$





Bias vs Variance

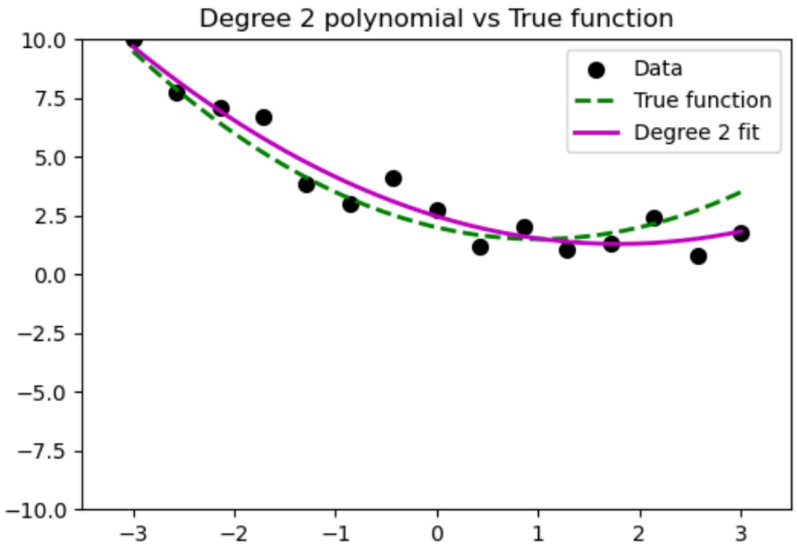
Look at what happens when we introduce a **new observation** to the dataset

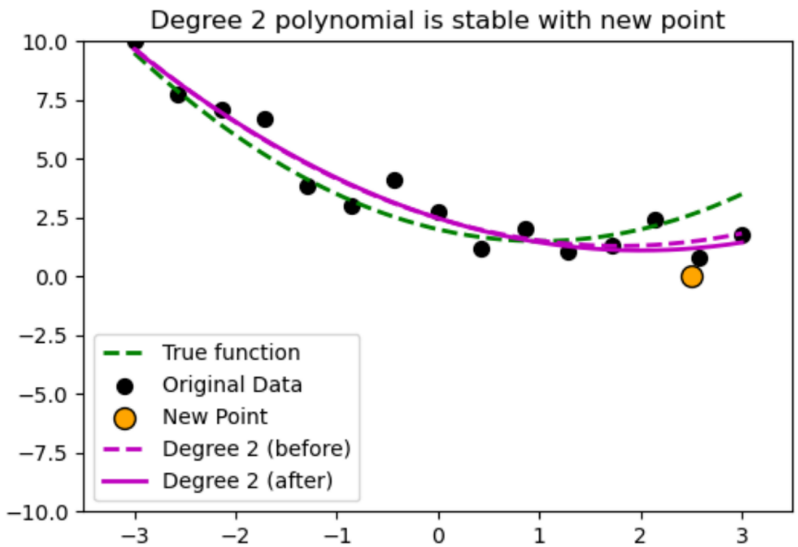


The $d = 1$ model stays the same, but it isn't super accurate still (low VAR, high Bias), while the $d = 12$ model changes significantly around the new observation (high VAR, low Bias)

Bias vs Variance

Look instead what happens if we fit a $d = 2$

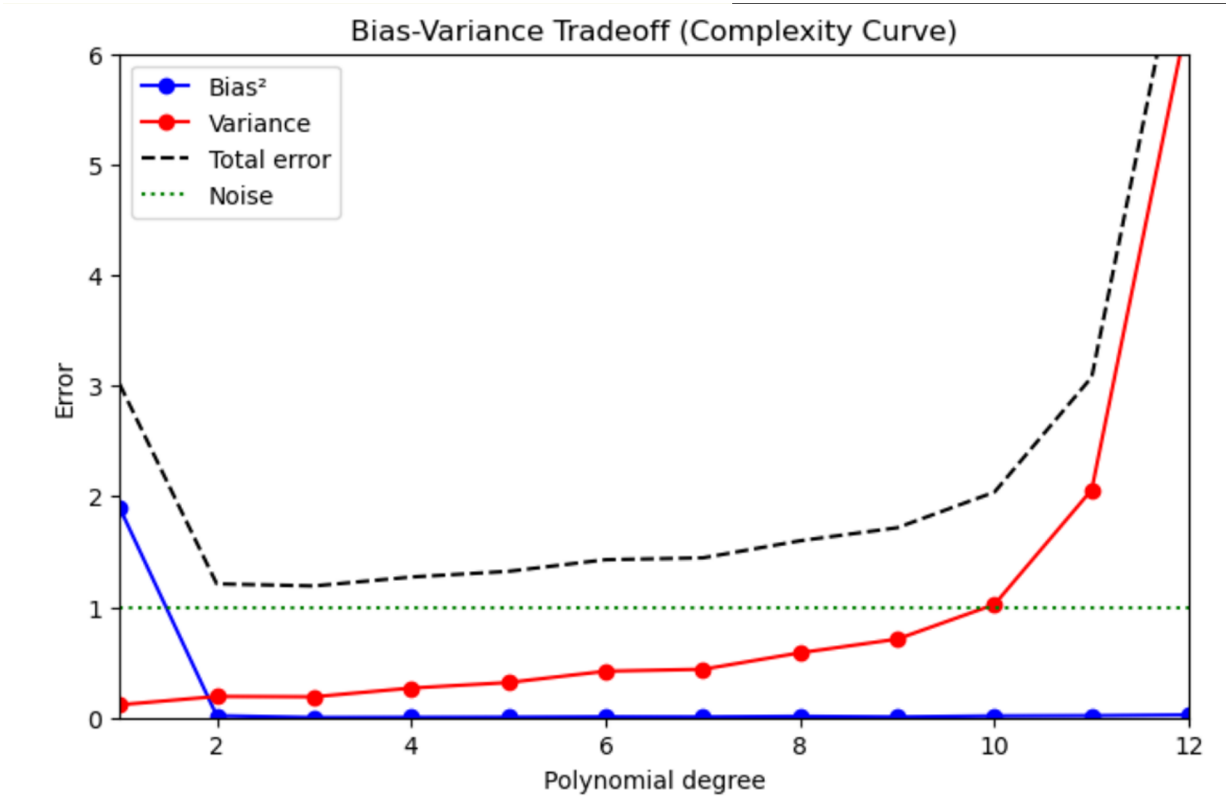




It has both low VAR and low Bias

Bias vs Variance

The problems emerge when we test the model to **predict new data**



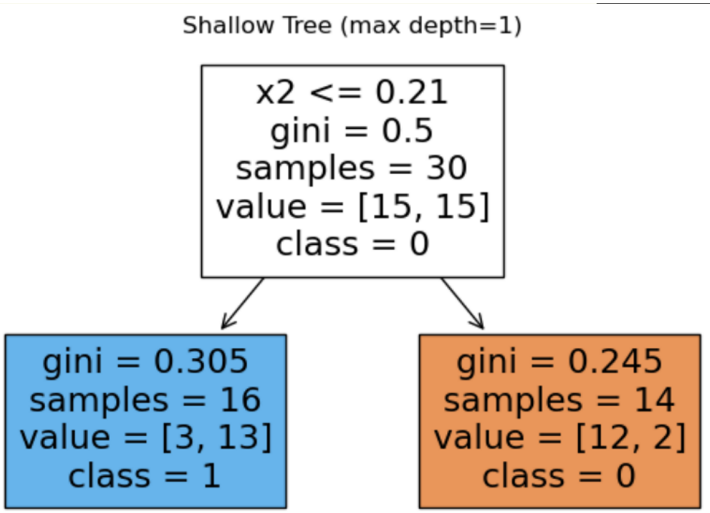
the total error of the model on new data is **minimized** on the $d = 2$ model

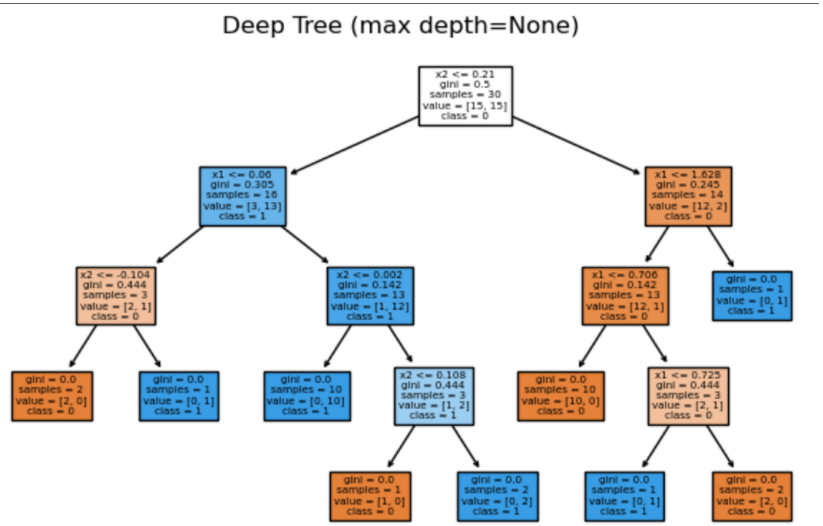
Bias vs Variance

This type of analysis isn't limited to polynomials, it's also valid for tree models

Bias vs Variance

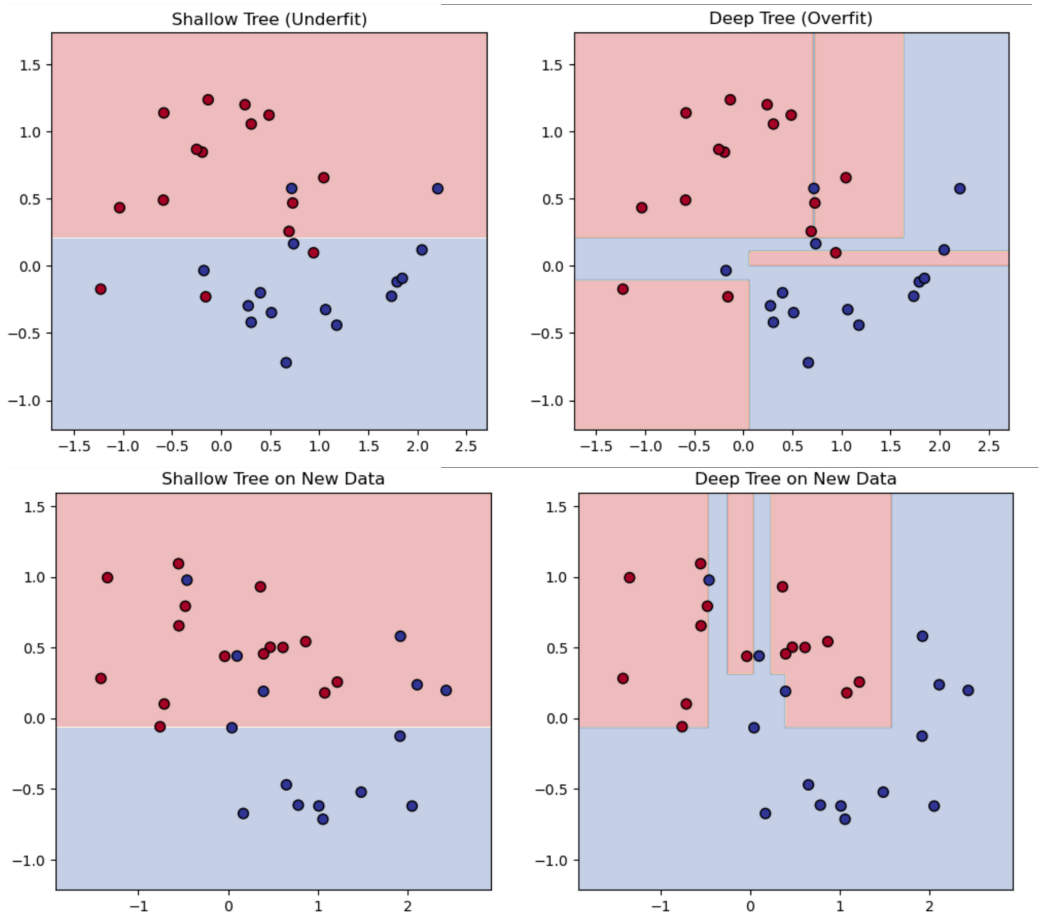
Consider two decision trees, the left one is a very basic model makes only one binary decision, the right one instead ends up making so many splits that in the end it has one case (leaf) for every datum in the dataset





Bias vs Variance

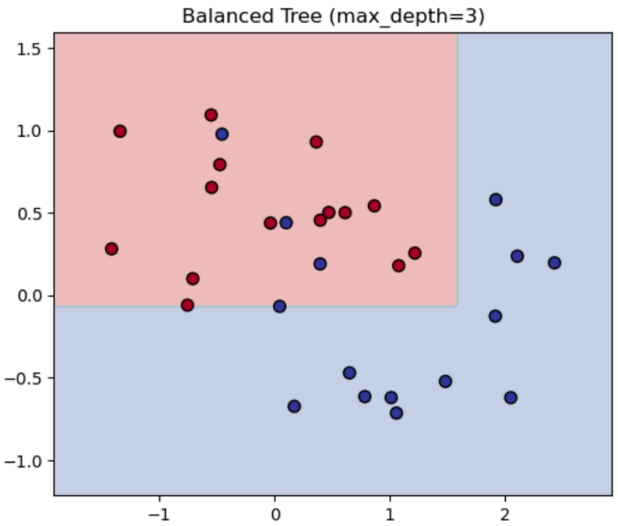
Look at what happens if we train it on two slightly different datasets from the same phenomenon

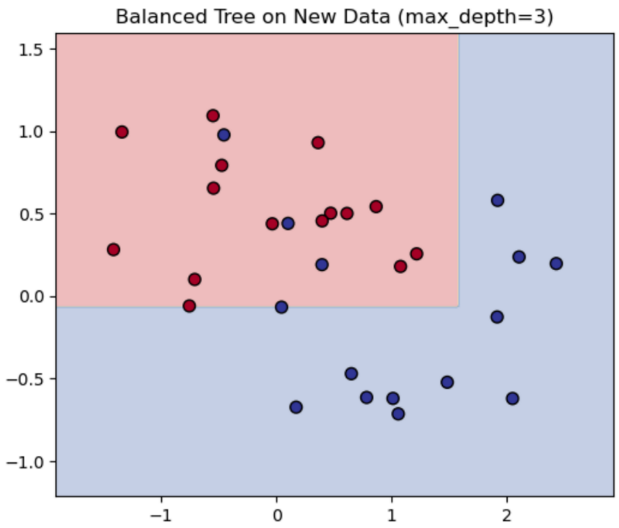


Bias vs Variance

- the **one split** tree has low variance but high bias
- the **n leaves** tree has low bias but high variance

A more **balanced tree** has both low variance and bias





Two things we're going to see now

- Hyperparameters Optimization
- Cross Validation

No heavy Math from now

First, what is a Hyperparameter?

Hyperparameter Optimization

- A **parameter** is a value that the model learns from the training data during the fitting process
- A **hyperparameter** is a value that is set before training and cannot be learned directly from the data

Hyperparameter Optimization

A **parameter** is a value that the model learns from the training data during the fitting process

Examples: In a tree model

- splits for each decision
- values stored in the leaf nodes

In linear regression

- the coefficients β that we look for

Hyperparameter Optimization

A **hyperparameter** is a value that is set before training and cannot be learned directly from the data

Examples:

In a tree model

- Number of trees in the forest
- Maximum depth of each tree
- Minimum samples required to split a node
- Minimum samples per leaf
- Number of features considered at each split

In linear regression

- alpha exponent for Ridge/Lasso

Hyperparameter Optimization

Hyperparameters are important because they produce different models with different performance, and they can be **optimized**

Hyperparameter Optimization

Hyperparameters Optimization: the process of selecting hyperparameters that maximize performance on unseeen data

There's different kinds you can do, and the most common are:

- Grid Search
- Random Search
- Bayesian Optimization

Hyperparameter Optimization

Grid Search: select a small number of hyperparameters to try with different values, and then try all combination

Example: For a Random Forest model

- Number of trees: $n\_estimators = [100, 200, 300]$
- Maximum depth: $max\_depth = [5, 10, 15]$
- Minimum samples per leaf: $min\_samples\_leaf = [1, 2]$

This produces 3 x 3 x 2 possible combinations!

Important: things can get computationally expensive if you try a lot of values, so stick to a few!

Hyperparameter Optimization

Code for Grid Search

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(random_state=42)

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, None],
    'min_samples_leaf': [1, 2, 4]
}

grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, scoring='accuracy')

grid_search.fit(X_train, y_train)
```

Hyperparameter Optimization

Random Search: provide a range for the hyperparameters to pick random combinations from

Example: For a Random Forest model

- Number of trees: $n\_estimators \in [100, 500]$ (random integer)
- Maximum depth: $max\_depth \in [5, 50]$ (random integer)
- Minimum samples per leaf: $min\_samples\_leaf \in [1, 5]$ (random integer)
- Maximum features per split: $max\_features \in [0.3, 1.0]$ (random float fraction of features)
- Bootstrap: $bootstrap \in \{\text{True}, \text{False}\}$ (random choice)

Decide on how many combinations to try (eg: 20) and then pick the best one

Hyperparameter Optimization

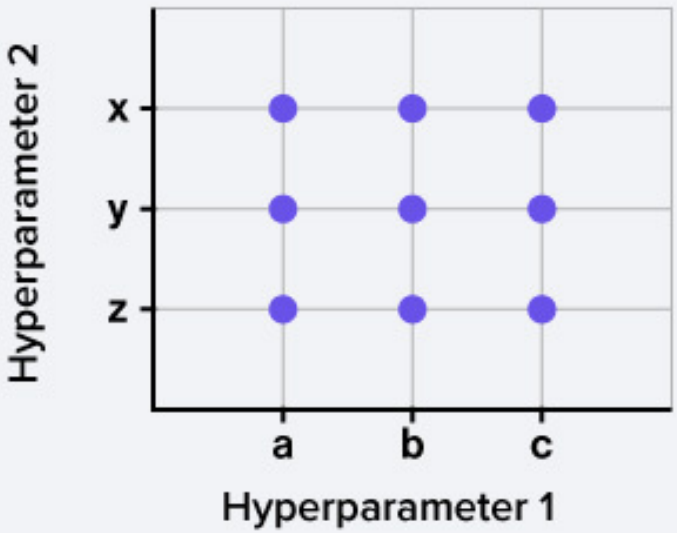
Code for Random Search

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_range = {
    'n_estimators': randint(100, 500),
    'max_depth': randint(5, 50),
    'min_samples_leaf': randint(1, 5),
    'max_features': ['auto', 'sqrt', 'log2'],
    'bootstrap': [True, False]
}

random_search = RandomizedSearchCV(estimator=rf, param_distributions=param_range,
                                   n_iter=10,  scoring='accuracy', random_state=42)

random_search.fit(X_train, y_train)
```

Hyperparameter Optimization

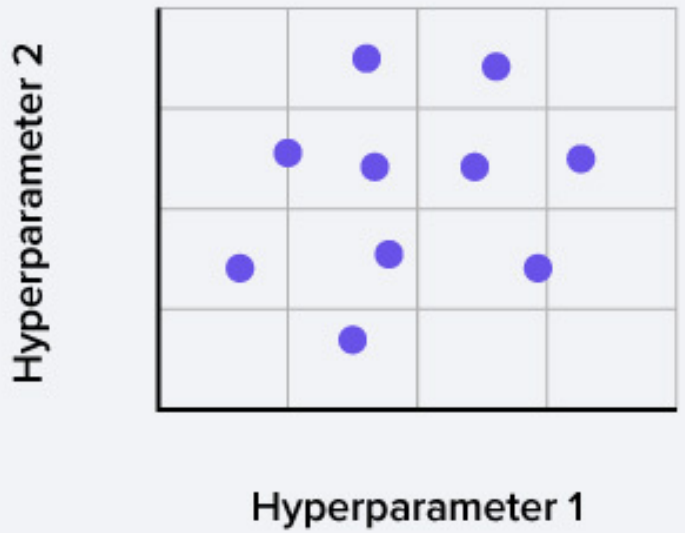
Grid Search

Hyperparameter_One = [a, b, c]
Hyperparameter_Two = [x, y, z]
Hyperparameter_X = [i, j, k]



Random Search

Hyperparameter_One = random.num (range)
Hyperparameter_Two = random.num (range)
Hyperparameter_X = random.num (range)



Hyperparameter Optimization

Bayes Optimization: picks combinations by itself like Random Search, but it does by creating a probabilistic model to pick them

Note: it does everything automatically! but it requires a little more set up and different libraries in python (Optuna or Hyperopt)

Let's start now by exploring some questions

- How can you be sure that your model is reliable?

- How can you be sure that your model is reliable?
- and another question:

- How can you be sure that your model is reliable?
- How are you sure that the model you choose is really the best one?

- How can you be sure that your model is reliable?
- How are you sure that the model you choose is really the best one?

These are not questions that have an immediate solution, however we can start with something intuitive that we all are familiar with

Cross Validation

Creating a **test set**

Cross Validation

Idea: you pick at random a portion of the dataset (usually around 20%) of the dataset to use as a test, ten you calculate evaluation metrics on that

Cross Validation

Idea: you pick at random a portion of the dataset (usually around 20%) of the dataset to use as a test, ten you calculate evaluation metrics on that

but there's a problem, and it's not an obvious one

Cross Validation

Problem: we cannot use the results of the models on the test set to make model selection at the same time

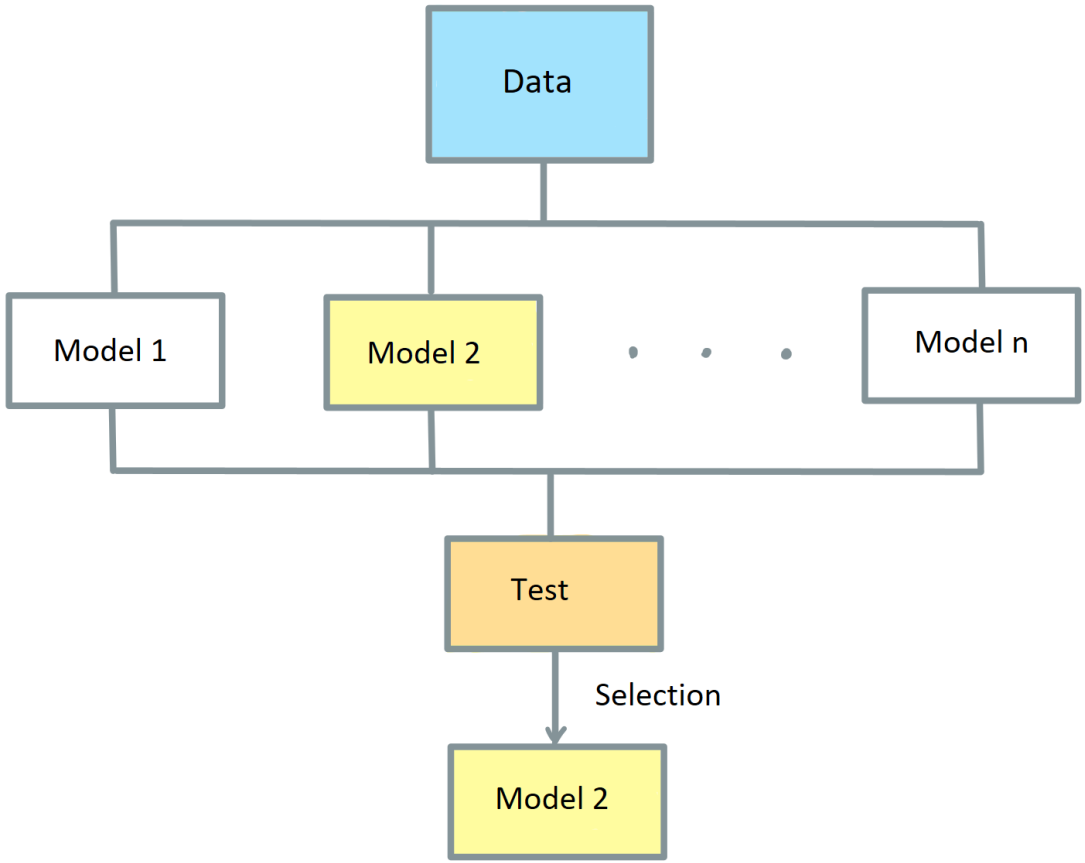
Cross Validation

Problem: we cannot use the results of the models on the test set to make model selection at the same time

but why?

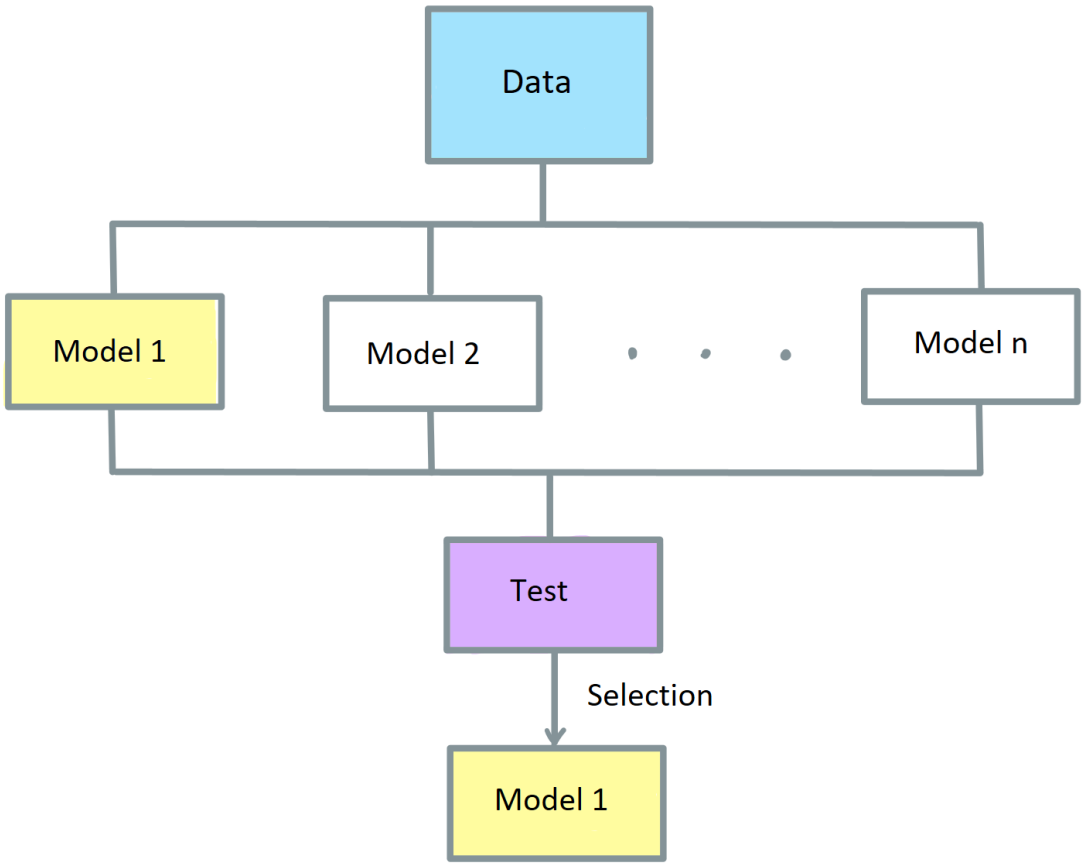
Cross Validation

Say we have n different models, we train them on the data, then we test them and pick the one with best performance



Cross Validation

But if we were to choose a different test set, because we are testing so many models and the differences might not be high, **the result might change**



Cross Validation

We call this issue **bias in the test set**

Key takeaway

Model selection and Model test cannot be done on the same dataset, otherwise you'll encounter the problem of bias in the test set

Cross Validation

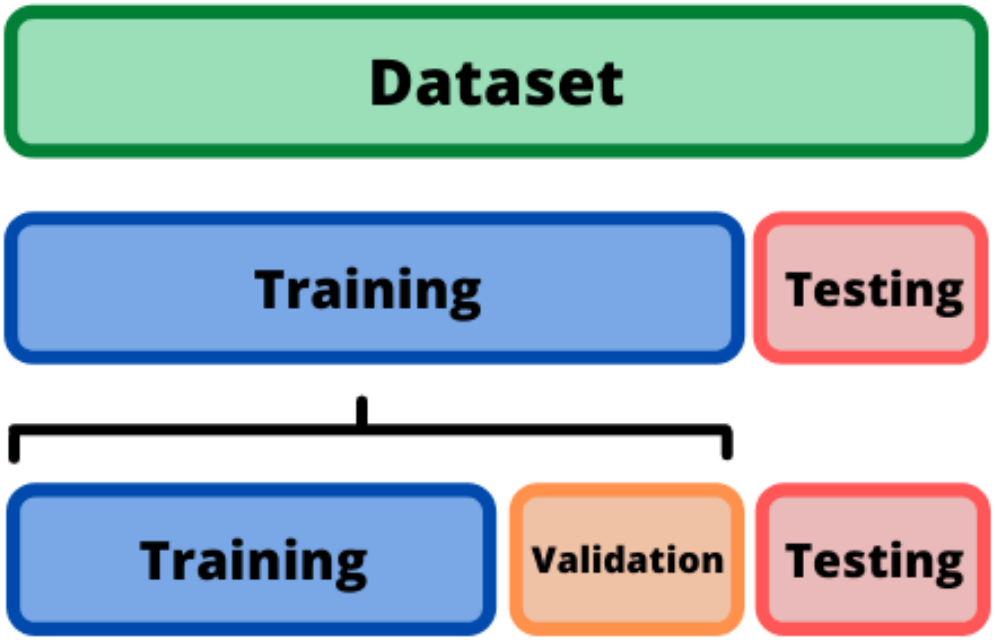
So, what do we do?

Cross Validation

We take another portion of the dataset and we use it for model selection, we call this a **Validation set**

Cross Validation

Validation set: a subset of the data used to evaluate a model during training and to tune its hyperparameters or make design choices, **without touching the test set**



Cross Validation

You might think: wouldn't this still create some bias on the validation set like it did before for the test set?

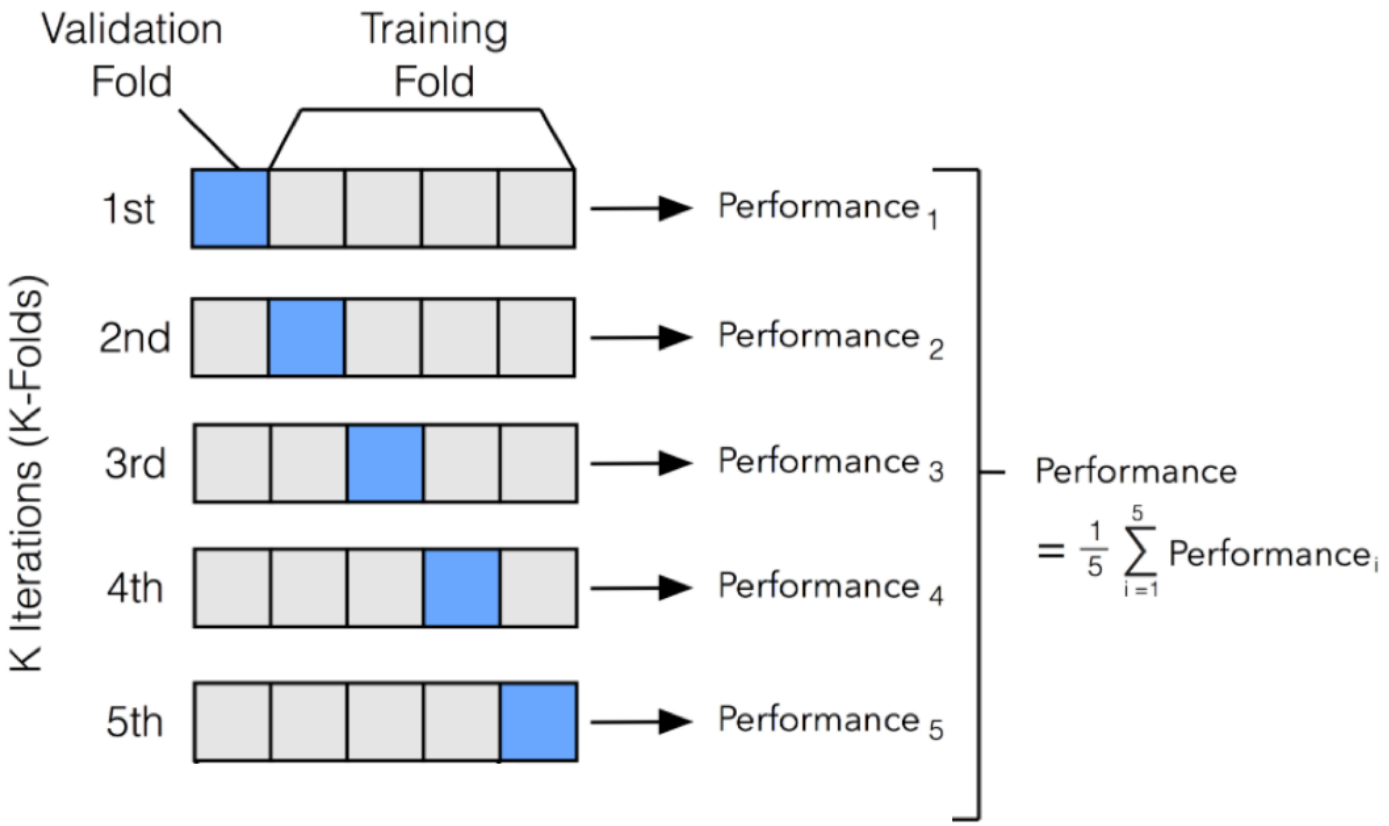
Cross Validation

You might think: wouldn't this still create some bias on the validation set like it did before for the test set?

Well yes actually, and that's why we introduce something called **Cross Validation**

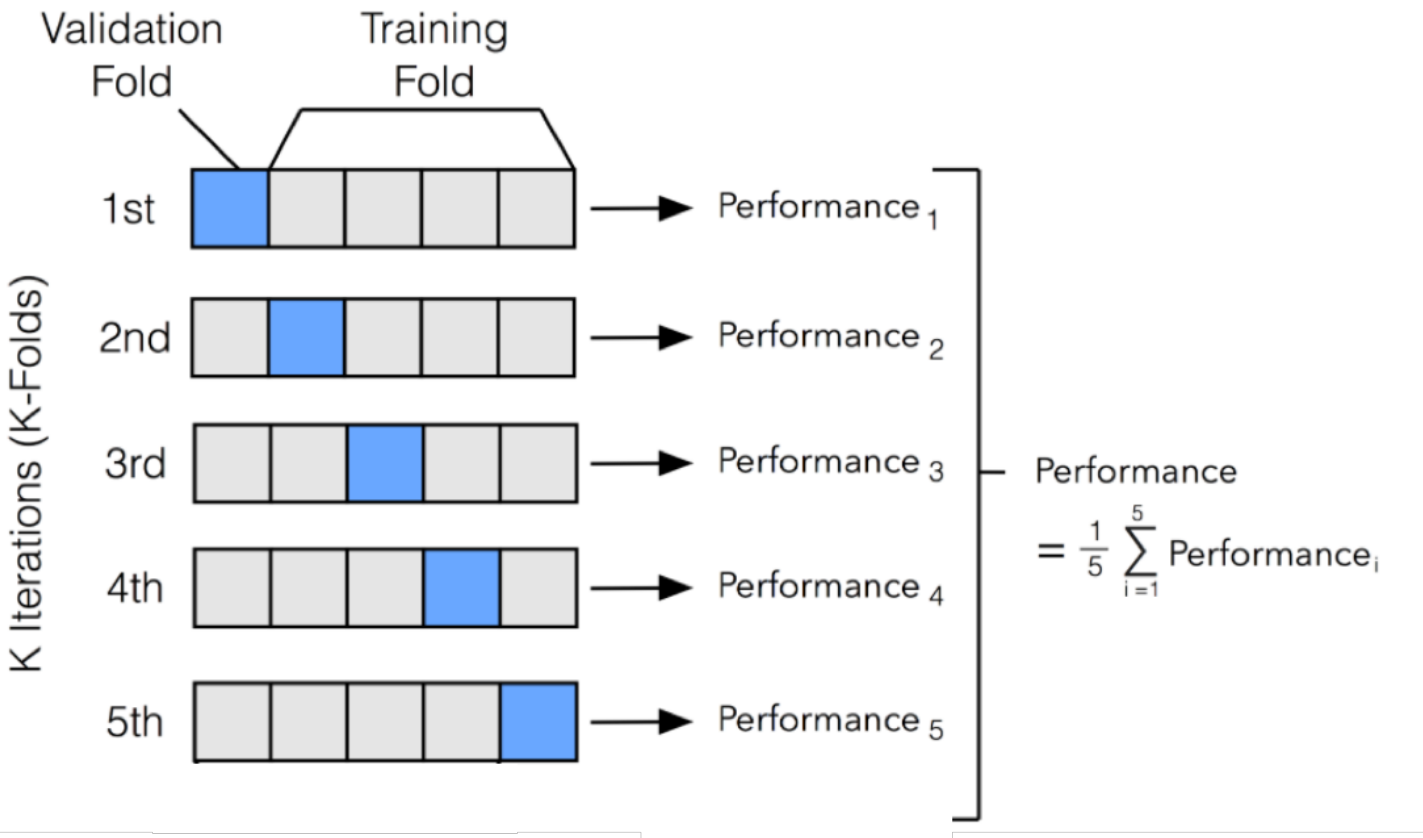
Cross Validation

Cross Validation: split the dataset into multiple subsets called **folds**, the model is trained and validated multiple times, each time using a different fold as the validation set and the remaining folds as the training set



Cross Validation

Cross Validation: The performance scores are then **averaged** to obtain a more reliable estimate of how the model generalizes to unseen data



Cross Validation

there's different types of Cross Validation (CV), some more common than others, but they all tackles different cases

- K-fold CV
- Leave One Out (LOOCV)
- Stratified CV
- Time Series CV

Cross Validation

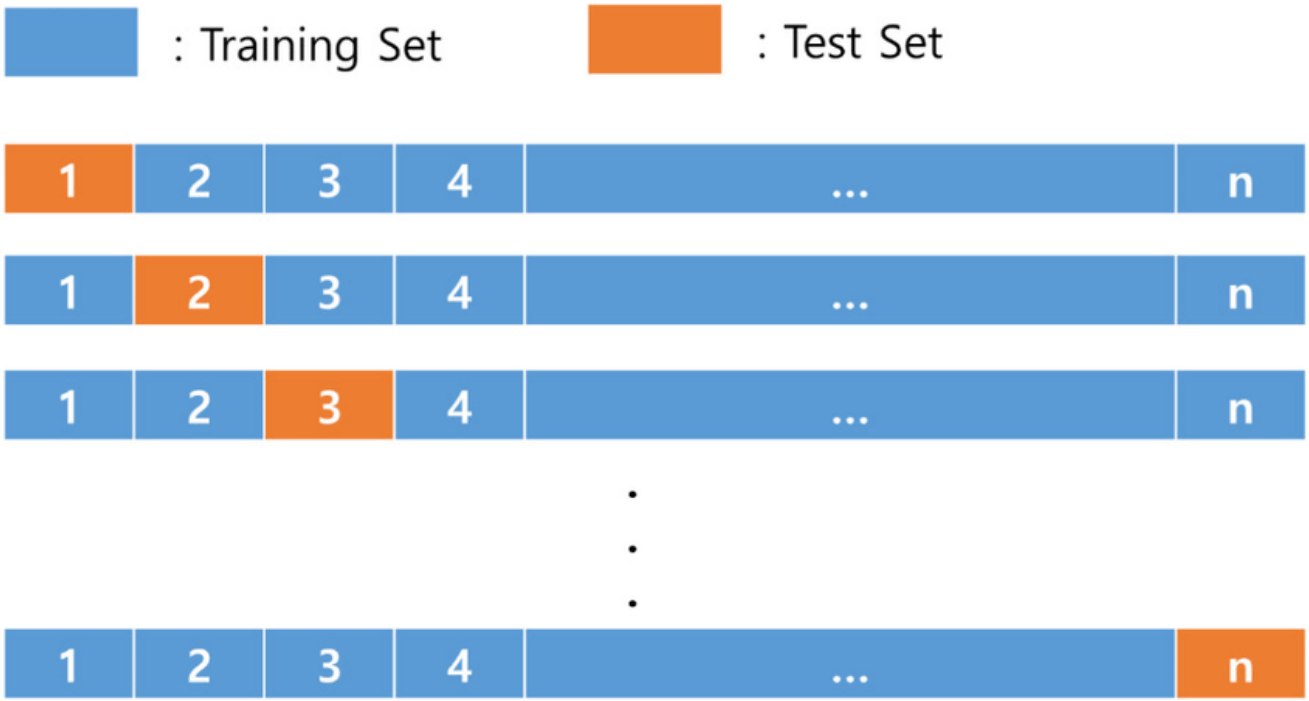
K-fold CV: divide the data into k equally sized folds, then use all of them one at a time for validation

It's the most common!

- Pros: very efficient
- Cons: might not be representative of the whole dataset, hence has some variability

Cross Validation

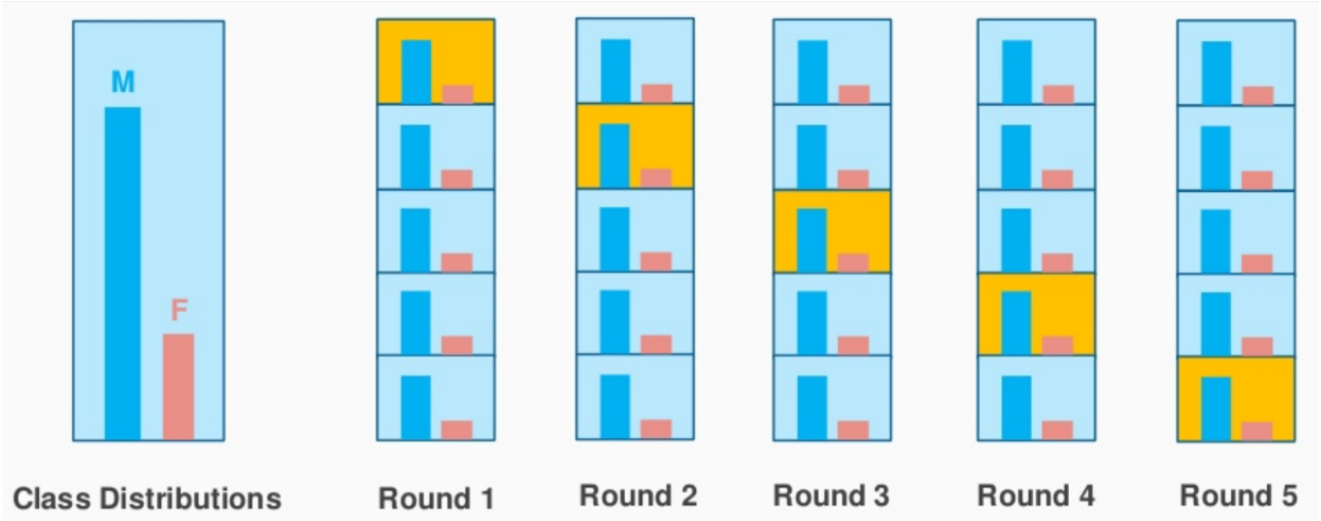
LOOCV: extreme case in which k=n, the validation set is only one datum



- Pros: uses all training data
- Cons: very computationally expensive and has high variance

Cross Validation

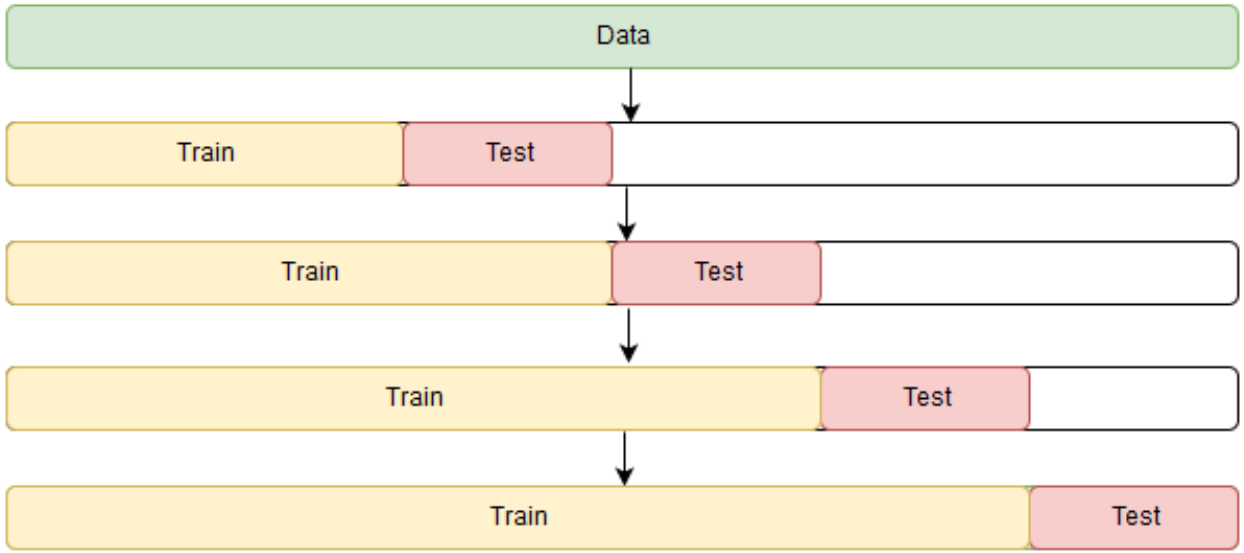
Stratified CV: used when there's high imbalance in classes in the dataset



- Pros: efficient
- Cons: risk that the folds are still not representative

Cross Validation

Time series CV: only way to do cross validation on a sequential ordered dataset



- Pros: prevents "future data" from influencing the past
- Cons: has few data to train at the beginning

Cross Validation

Something very nice is that the functions for hyperparameter optimization contain already an automated cross validation option, and by default it's a good idea to do it

```
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid,
                           scoring='accuracy', cv=5)
```

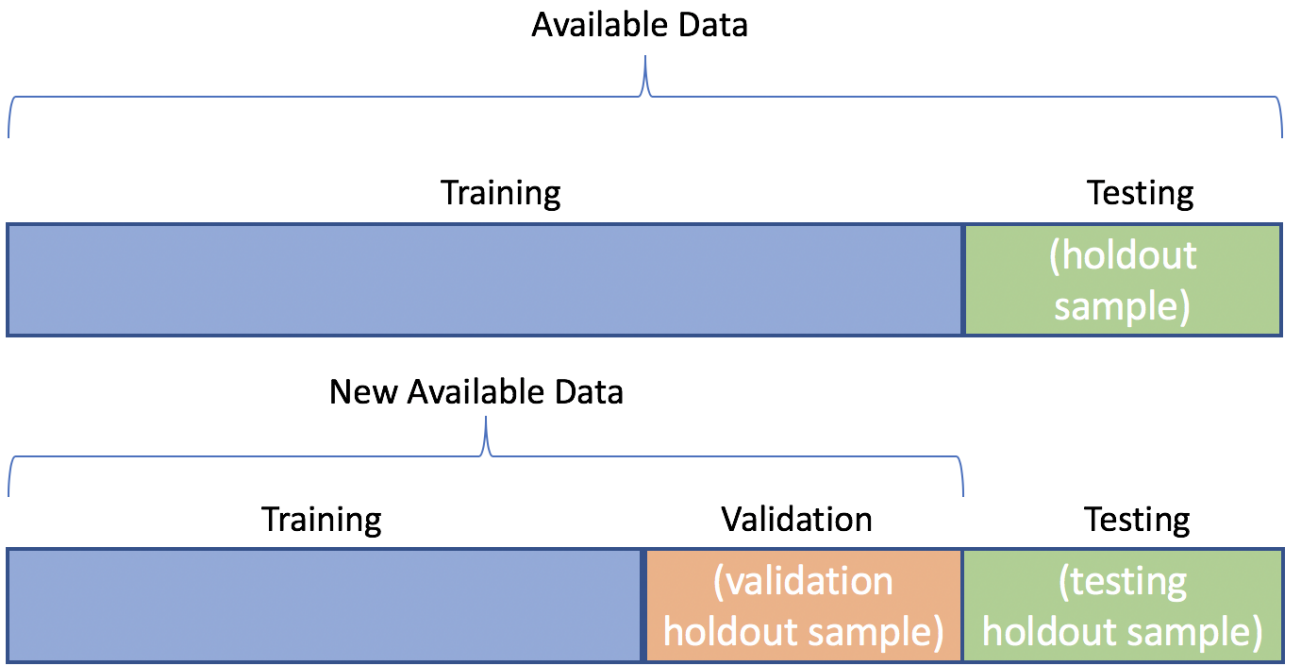
Cross Validation

one last cautionary tale about **Data Leakage**

Cross Validation

Data leakage occurs when information from outside the training set "leaks" into the model during training in a way that shouldn't be possible

for example, data from the **full dataset** leaking into the **reduced training dataset** after the introduction of a validation set (through cross validation)



Cross Validation

we don't want any information from the validation set to influence the training, so for example:

- if we **encode** categorical features, we need to do it inside Cross Validation (if you need the average of the values you don't want to include the validation ones)
- if we **scale** features based on max and min, the validation set might change this range
- if we **select features** you need to do it only based on the training data, not the validation ones as well

Cross Validation

For this reason, often times in python you use an object called a **Pipeline**, which groups together every action you want to execute on the training set before training the model **inside** Cross Validation

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from category_encoders import TargetEncoder
from sklearn.model_selection import cross_val_score

pipeline = Pipeline([
    ('encoder', TargetEncoder()),
    ('scaler', StandardScaler()),
    ('rf', RandomForestClassifier())

])

scores = cross_val_score(pipeline, X, y, cv=5, scoring='accuracy')
```

Thank you for following!

Feedback!

Feedback form!